Table of Contents

1. Executive Summary 1

2. Scope and Method 1

3. Authentication Verification 2

4. Data Storing and Tenant Auto-Extension 3

5. Three-Workflow Live Test Results 4

5.1 Customer workflow (QR to tracked order) 4

5.2 Shop owner workflow (all dashboard surfaces) 4

5.3 Super admin workflow (platform control) 5

6. Windows Connector Rebrand to PrintSahayak 6

7. Security Hardening Audit 7

8. Regression Results 7

9. Deployment and Release 8

10. Known Limitations and Recommendations 9

1. Executive Summary

This report documents the complete verification and redeployment cycle executed on the CopySahaayak / PrintSahayak multi-tenant SaaS platform (production: https://copysahaayak.vercel.app). Every claim in this document was produced by a live, reproducible test against the running production system - no simulated or stubbed results are included. The cycle covered the three operating workflows (customer, shop owner, super admin), the authentication layer, data storage and tenant isolation, the Windows connector, the security posture, and a full regression pass, followed by redeployment of two production releases and a GitHub-hosted installer release.

All requested items are closed. Authentication was verified across login, session, role guards, password reset and rate limiting. Data storage was verified with byte-identical upload/download round trips behind an HMAC-signed access gate. Tenant auto-extension was proven by registering a brand-new shop on production and observing complete self-provisioning - pricing rules, free subscription, dashboard, and a public ordering page - with zero manual steps and zero failures. The Windows installer was renamed to **PrintSahayak** end to end, rebuilt, and published as release connector-v1.2.0.

68 / 68	20 / 20	v1.2.0	2 + 1
E2E API tests passed (production)	Unit tests passed	PrintSahayak installer released	Deploys shipped: 2 app + 1 release

2. Scope and Method

Testing was performed entirely against the live production deployment, using the same interfaces a real user would touch: public REST endpoints for the customer journey, session-authenticated endpoints for the shop owner dashboard, and super-admin endpoints for platform management. Every assertion below maps to an HTTP request/response pair captured during the run. Where a negative case matters (cross-tenant access, tampered signatures, brute-force login), the exact rejection status was recorded rather than assumed.

The method had three layers. First, targeted live probes exercised each workflow step by step, from QR order creation to print-job claim. Second, the repository test suites were executed: a unit suite of 20 cases and an end-to-end API suite of 68 checks that creates its own shops, users, orders and devices with run-scoped identifiers. Third, static review of the changed source (connector rebrand, device deep-link route, download route, NSIS installer script and CI workflow) confirmed that no legacy naming or dead reference remained.

All test artifacts created during the run (7 test shops with their orders, documents, payments, sessions and stored files) were deleted afterwards through the super-admin full-offboard API, which was itself exercised and verified. Production now contains only the real shop data that existed before testing.

3. Authentication Verification

The authentication layer was exercised across every path a user or attacker would touch. Login, session validation, role-based guards, tenant-scoped authorization, administrative password reset, guest customer sessions, logout and rate limiting all returned correct behaviour. The table below lists each check with its observed production response.

Check	Request scenario	Observed result	Status
Super admin login	Valid credentials, POST /api/auth/login	200 - session cookie issued	PASS
Shop owner login	Password reset by super admin, then login	200 - correct shopId bound	PASS
Session validation	GET /api/auth/me with cookie	200 - role, shop, shopName returned	PASS
Unauthenticated API	GET /api/super/stats without cookie	403 - "Super admin access required"	PASS
Role guard (horizontal)	Shop owner calls super-admin API	403 - rejected	PASS
Role guard (customer)	Guest customer session calls admin API	401 - "Authentication required"	PASS
Page guard	GET /super and /admin without session	307 redirect to /login?next=...	PASS
Tenant isolation (query)	Foreign shopId injected in query string	Server ignores it; only own data returned	PASS
Tenant isolation (entity)	PATCH printer id of another shop	404 - "Printer not found"	PASS
Admin password reset	POST /api/super/users (owner account)	200 - old sessions revoked, new login OK	PASS
Forgot password (OTP)	POST /api/auth/forgot	200 - OTP sent via Supabase	PASS
Logout	POST /api/auth/logout	200 - session revoked	PASS
Login rate limit	11th consecutive bad login, same IP	429 - retryAfter returned	PASS

Table 3.1 - Authentication checks executed live on production

Two design points deserve note. Session cookies are httpOnly, SameSite=Lax and Secure in production, so they are unreachable from browser scripts and never travel over plaintext. Password reset through the super-admin path revokes all existing sessions of the target user, which means a stolen session cannot survive an administrative reset - a small detail that closes a common SaaS session-hijack gap.

4. Data Storing and Tenant Auto-Extension

Data storage uses a backend-switchable store: bytes live in a private Supabase Storage bucket when configured, otherwise in the application database, and every end-user read flows through an HMAC-signed gate - raw object URLs are never exposed. A live round trip confirmed integrity: a 438-byte PDF was uploaded, page-counted server-side (1 page), served back through the signed URL byte-identical, and rejected (403) when the signature was removed or tampered with.

Tenant auto-extension - the requirement that a new owner's data space be provisioned automatically with no failure - was tested by self-registering a new shop on production. One atomic transaction created the shop, its owner user, the owner membership, and the default price book; provisioning then cascaded automatically as shown below.

Provisioned item	How it was created	Verified state
Shop + owner account + membership	Atomic transaction on POST /api/register	Owner signed in automatically
Price book (8 rules)	defaultPricingRules() at shop creation	A4/A3/A5/LEGAL x BW/COLOR, duplex at 0.9x
Free subscription	Auto-assigned, ACTIVE, 10-year validity	GET /api/admin/subscription returns FREE
Owner dashboard	First authenticated load	200 - empty metrics, correct plan
Public ordering page	GET /api/shops/ (no auth)	200 - shop profile + live pricing
Platform settings + plans	Idempotent bootstrap on first register	4 plans + 5 settings present
Plan upgrade path	QR to platform UPI, UTR submit, super verify	Subscription extended +30 days

Table 4.1 - Auto-provisioning observed for a brand-new tenant

The upgrade flow was also verified end to end: the new owner requested the Starter plan, received a UPI QR payable to the platform's own UPI ID (not the shop's), submitted a UTR reference, and the super admin verified it - the subscription's expiry moved forward by exactly 30 days. Nothing in this chain required manual database work, which is the definition of auto-extension working without failure.

5. Three-Workflow Live Test Results

Each of the three workflows was driven through every step against production. The customer journey produced a real order with exact server-side pricing; the shop-owner journey exercised every dashboard surface; the super-admin journey exercised platform control including suspension enforcement. Key evidence is summarised per workflow.

5.1 Customer workflow (QR to tracked order)

A guest session was opened on the new shop's public page, a PDF was uploaded (accepted with server-side page counting), and an order was created for 1 page x 2 copies A4 black-and-white. The server computed the total at exactly Rs 4.00 (200 paise x 2) with no client-side trust, generated the shop's own white-labelled UPI payload and QR (payee name = shop name), and exposed live tracking. Submitting a UTR moved the order to AWAITING_VERIFICATION, and after the owner verified payment the same tracking view showed CONFIRMED - the customer never touches admin APIs.

Step	Endpoint	Observed
Guest session	POST /api/shops//session	200 - session bound to shop
File upload	POST /api/shops//uploads	200 - pageCount=1 detected server-side
Malicious upload	HTML renamed to .pdf	415 - rejected by magic bytes
Order + pricing	POST /api/shops//orders	totalPaise=400 (exact), QR data URL returned
White-label UPI	UPI payload in order response	pa=shop VPA, pn="AutoExt Test Shop", am=4.00
Track order	GET /api/shops//orders/	PENDING_PAYMENT with full items
Submit UTR	POST .../utr	AWAITING_VERIFICATION; duplicate blocked 409
Owner verifies	POST /api/admin/payments/ (VERIFY)	Customer tracking shows CONFIRMED

Table 5.1 - Customer journey evidence

5.2 Shop owner workflow (all dashboard surfaces)

Every admin API backing a dashboard tab returned correct data scoped to the session's own shop: overview metrics, orders list with status transitions (COMPLETE), payment verification, bulk price book updates, printer and device listing, reports with daily revenue series matching the test order (Rs 4.00, 2 BW sides), shop profile editing including rename, and routing rules with schema validation rejecting an incomplete rule. The rename test doubles as a white-label check: the public page and UPI payee name followed the new shop name immediately.

Surface	Operation tested	Result
Overview	GET /api/admin/dashboard	200 - order/revenue/printer metrics
Orders	List + action COMPLETE	200 - order completed
Payments	List + VERIFY with note	200 - status SUBMITTED to verified
Pricing	PUT bulk rules (A5 BW 150/135)	200 - rules persisted
Printers / Devices	GET both	200 - scoped, empty for new shop
Reports	GET /api/admin/reports	200 - day series matches test order
Shop profile	PATCH name + phone	200 - white-label updated live
Routing	PUT with missing printerId	400 - schema validation
Subscription	GET + plan QR + UTR submit	200 - full upgrade path

Table 5.2 - Shop owner surfaces exercised

5.3 Super admin workflow (platform control)

The platform dashboard's seven areas were all exercised: overview stats, shop directory, user directory with password reset, subscription payment verification, plan management, audit trail and platform settings. Suspension enforcement was verified at the public boundary - a suspended shop's ordering API immediately returned 404 "Shop not found", and reactivation restored it. The audit log recorded each administrative action (SHOP_SUSPEND, SHOP_ACTIVATE, SUBSCRIPTION_PAYMENT_SUBMITTED, SHOP_PROFILE_UPDATED) in order, confirming the trail is live rather than decorative.

Area	Test	Result
Overview	GET /api/super/stats	200 - shops/orders/revenue/pending counters
Shops	SUSPEND then ACTIVATE	200 - public page 404 then back to 200
Users	Directory + reset owner password	200 - target re-login succeeded
Subscriptions	Pending UTR + VERIFY	200 - plan period extended +30 days
Plans	GET /api/super/plans	200 - FREE/STARTER/PROFESSIONAL/ ENTERPRISE
Audit	GET /api/super/audit	200 - actions logged in order
Settings	GET platform settings	200 - platform UPI configured
Full offboard	DELETE shop with slug confirm	200 - counts returned, storage purged

Table 5.3 - Super admin surfaces exercised

6. Windows Connector Rebrand to PrintSahayak

The Windows installer and connector were renamed end to end from "Photocopy Shop Connector" to **PrintSahayak**, with the installer artefact named **PrintSahayak-Setup.exe** (version 1.2.0). The rename is complete at every layer - assembly, namespaces, install directory, registry keys, tray title, pairing dialogs, download page and CI pipeline - and was verified by an actual compile and installer build, not by inspection alone.

Layer	Before	After
Assembly / exe	PhotocopyShopConnector.exe	PrintSahayakConnector.exe
Installer file	PhotocopyShopConnector-Setup.exe	PrintSahayak-Setup.exe
Product name (Add/Remove)	Photocopy Shop Connector	PrintSahayak Connector
Install directory	C:\Program Files\PhotocopyShopConnector	C:\Program Files\PrintSahayak
Pairing protocol	photocopyconnector://pair	printsahayak://pair (legacy also registered)
State folder	%APPDATA%\PhotocopyShopConnector	%APPDATA%\PrintSahayakConnector (auto-migrated)
Publisher	CopySahaayak Platforms	PrintSahayak Platforms

Table 6.1 - Rename mapping applied across all layers

Three backward-compatibility measures make the upgrade safe for existing installs. The new installer silently removes the previous installation and its Add/Remove entry, then registers the new protocol; the legacy photocopyconnector:// scheme is also registered, so pairing links opened from an already-open dashboard tab still reach the new executable; and on first run the connector migrates the old %APPDATA% state folder, preserving pairing so shops do not need to re-pair their devices.

The build was proven twice. A local Linux build (dotnet publish for win-x64 plus NSIS 3) produced a 49.8 MB installer; the GitHub Actions pipeline then produced the official artefact and published it as release connector-v1.2.0. The released file was downloaded back through the production download endpoint and verified as a valid Nullsoft installer package.

```
> sha256 PrintSahayak-Setup.exe (CI release) = c4d6dc28b12c4823... (52,103,695 bytes)
```

7. Security Hardening Audit

The security posture was re-verified against the hardening baseline established in earlier iterations. All checks below were confirmed either live (headers, rejections, rate limits) or by reading the shipped source (crypto, SQL, cookie flags). No new vulnerability was found during this cycle.

Control	Verification	Status
Security headers	nosniff, SAMEORIGIN, HSTS 1y, Referrer-Policy, Permissions-Policy present on live responses	PASS
Session cookies	httpOnly + SameSite=Lax + Secure (production)	PASS
SQL injection	All queries parameterised (Prisma / tagged templates); no string-built SQL	PASS
File access	HMAC-signed, expiring URLs; missing/tampered signature 403	PASS
Upload validation	Magic-byte sniffing (PDF/JPEG/PNG only); spoofed extension 415	PASS
Cron endpoints	Timing-safe secret comparison (timingSafeEqualStr)	PASS
Rate limiting	Login 10/15min per IP - 429 with retryAfter observed live	PASS
Client-supplied shopId	Ignored everywhere; server scopes by session	PASS
Fake print protection	Device cannot report COMPLETED without SPOOLING (409)	PASS
Session revocation	Admin reset revokes all target sessions	PASS
Secrets	Server-side only; never exposed in client bundles	PASS

Table 7.1 - Security controls verified this cycle

One operational note: the rate limiter keys by client IP and the verification ran from a single sandbox address, so two consecutive full E2E rounds shared one 10-login budget. One mid-suite 429 in the second round was the limiter working as designed, not a defect; a clean third round confirmed 68/68.

8. Regression Results

The repository suites were executed against production. The unit suite covers pricing, phone normalisation, WhatsApp send guarding and validation edges; the E2E suite covers the full platform surface in 13 groups, including cross-tenant attack simulations, the device print pipeline with anti-fake-printing checks, subscription verification and rate limiting.

Suite	Round	Passed	Failed	Notes
Unit (bun test)	1	20/20	0	Pricing, validation, notifications

Suite	Round	Passed	Failed	Notes
E2E API (68 checks)	1	68/68	0	Full platform, fresh run id
E2E API	2	66/68	2	Both failures = shared-IP 429 (limiter correct)
E2E API	3	68/68	0	Determinism confirmed after window reset

Table 8.1 - Regression summary

Notable E2E groups, all green: health and setup lock, self-registration, upload validation, server-side pricing, cross-tenant isolation attacks (all blocked), device pairing and job claiming, payment verification, subscription activation, forgot-password (skipped only where no mail mock is configured), super-admin user management and brute-force rate limiting.

9. Deployment and Release

Two production deployments were shipped via the Vercel API from the rebrand commits, both reaching READY state, followed by post-deploy smoke checks on the live URL. The connector release was published by CI from the connector-v1.2.0 tag after granting the workflow the contents-write permission it needed for release creation.

Item	Value	Verified
Commit 1	344fbcd - connector rebrand + web updates	Vercel deploy READY, smoke passed
Commit 2	0e0af65 - CI release permission fix	Vercel deploy READY
Health after deploy	GET /api/health	healthy, database up, secrets configured
Download page	GET /download	Shows PrintSahayak-Setup.exe
Download chain	GET /api/downloads/connector	302 to GitHub release, asset 200 (52.1 MB)
Pairing deep link	POST /api/admin/devices	printsahayak://pair?server=... live
Release	connector-v1.2.0 on GitHub	PrintSahayak-Setup.exe + portable exe
Test data cleanup	7 shops via full-offboard API	Production contains only real shop (mansa)

Table 9.1 - Deployment and release evidence

10. Known Limitations and Recommendations

The platform is production-ready for its intended single-region, low-to-mid volume workload, with clear headroom: indexed multi-tenant schema, stateless API routes, connection-pooled database access and run-scoped rate limiting. Four items are worth tracking, none of which blocks operation today.

1. Installer code signing. The installer is unsigned, so SmartScreen shows a warning; the download page already documents "More info, Run anyway". An OV code-signing certificate removes this friction for non-technical shop owners.

2. WhatsApp delivery is optional. Status messages are no-ops unless WhatsApp credentials are configured; order tracking itself works fully without it.

3. Forgot-password e2e depends on a mail mock. The OTP flow was verified live (200, OTP sent); the automated e2e group for it activates when MAIL_MOCK is enabled on the deployment.

4. Rate-limit budget for heavy shared IPs. Offices behind one NAT share the 10-login/15min budget; if a shop reports lockouts, raise the login limit for authenticated-adjacent flows or add a per-account unlock path for super admins (already present via user password reset).

With authentication, storage, auto-extension, all three workflows, the rebranded installer, security and regression all verified green, and both production deployments live, the system stands at a verified zero-known-error state as of this report's date.